

# Frontend Baglanti ve React Mulakat Soru-Cevap Notlari

Bu dokuman, HRCore projesinde frontend tarafinin Supabase backend ile nasil baglandigini, React mimarisinin nasil kurulduğunu ve isveren tarafından sorulabilecek teknik sorulara verilecek cevapları icermektedir.

## 1. Bu projede frontend tarafinda hangi teknolojileri kullandin?

### Cevap:

Frontend tarafinda React, TypeScript, Vite ve React Router kullandim. React ile component tabanlı arayuz gelistirdim. TypeScript ile veri tiplerini daha kontrollu hale getirdim. Vite sayesinde gelistirme ortamı hızlı çalıştı. React Router ile login, dashboard ve modul sayfaları arasındaki route yapısını kurdum.

Backend bağlantısı için Supabase JS Client kullandım.

## 2. React uygulaması Supabase'e nasıl bağlanıyor?

### Cevap:

Supabase bağlantısı `src/lib/supabaseClient.ts` dosyasında merkezi olarak kuruldu. `.env.local` içindeki `VITE_SUPABASE_URL` ve `VITE_SUPABASE_ANON_KEY` değerleri okunuyor, sonra `createClient()` ile Supabase client oluşturuluyor.

Bu client daha sonra auth, employee, documents, devices, leave ve salary service dosyalarında kullanılıyor.

Akis şu şekilde:

```
React component
-> feature service
-> supabase client
-> Supabase API
-> PostgreSQL / Auth / Storage
```

## 3. Supabase sorgularını neden direkt componentlerin içinde yazmadın?

### Cevap:

Componentlerin içine direkt Supabase sorgusu yazsaydım UI kodu ile veri erişim kodu birbirine karıştırdı. Bu da projeyi büyüdükçe bakımı zor bir hale getirdi.

Bu yüzden her iş modulu için ayrı service dosyaları oluşturdum. Örneğin:

```
EmployeesPage -> employeeService -> employees tablosu
DocumentsPage -> documentService -> Storage + documents tablosu
DevicesPage -> deviceService -> devices + device_assignments tabloları
```

Boylece sayfalar sadece ekrani ve kullanıcı etkileşimini yönetiyor. Veri çekme, insert, update, delete ve hata yakalama gibi işlemler service katmanında duruyor.

#### 4. Bu service katmanı frontend mimarisi açısından ne kazandırıyor?

**Cevap:**

Service katmanı UI ile veri erişimini ayırıyor. Bu sayede:

- Component kodları daha temiz kalıyor.
- Supabase sorguları tek yerde toplanmış oluyor.
- Aynı veri fonksiyonu farklı sayfalarda tekrar kullanılabilir.
- Hata yönetimi daha düzenli oluyor.
- İleride Supabase yerine farklı backend gelirse sayfaları komple değiştirmek yerine service katmanı güncellenebilir.

#### 5. Projede klasör yapısını nasıl organize ettin?

**Cevap:**

Projede modüler bir yapı kullandım:

```
src/app      -> uygulama girişi ve router
src/pages    -> route seviyesindeki sayfalar
src/features  -> iş modüllerinin service ve logic dosyaları
src/components -> ortak UI ve layout componentleri
src/layouts  -> uygulama iskeleti
src/lib       -> supabaseClient ve yardımcı fonksiyonlar
src/styles   -> global stiller
```

Bu yapı sayesinde her modülün sorumluluğu daha net oldu.

#### 6. `pages` ve `features` klasörleri arasındaki fark nedir?

**Cevap:**

`pages` klasörü kullanıcının route olarak gördüğü ekranları tutar. Örneğin `EmployeesPage` , `DocumentsPage` , `DashboardPage` .

`features` klasörü ise bu sayfaların kullandığı iş mantığını ve Supabase servislerini tutar. Örneğin `employeeService` , `documentService` , `leaveRequestService` .

Yani:

```
pages = ekrani gösterir
features = veri ve iş mantığını yönetir
```

#### 7. Auth state frontend'de nasıl yönetiliyor?

**Cevap:**

Auth state `AuthProvider` icinde yönetiliyor. Uygulama acildiginda `supabase.auth.getSession()` ile mevcut oturum kontrol ediliyor. Login veya logout oldugunda `supabase.auth.onAuthStateChange()` ile degisimler dinleniyor.

Session bilgisi, user bilgisi, profile bilgisi ve login/logout fonksiyonlari React Context ile uygulamaya saglaniyor. Componentler bu bilgilere `useAuth()` hook'u ile erisiyor.

## 8. Neden auth icin React Context kullandin?

### Cevap:

Auth bilgisi uygulamanin bircok yerinde gerekli. Header, Sidebar, ProtectedRoute ve sayfalar kullanicinin login durumunu veya rolunu bilmek zorunda.

Bu bilgiyi prop olarak her yere tasimak yerine React Context kullandim. Boylece merkezi bir auth state olustu ve ihtiyac duyan componentler `useAuth()` ile bu state'e erisebildi.

## 9. Kullanici login olduktan sonra frontend'de ne oluyor?

### Cevap:

Login formu email ve password'u `signIn()` fonksiyonuna gonderiyor. Bu fonksiyon Supabase Auth'a `signInWithPassword()` ile istek atiyor.

Basarili login sonrasi Supabase session olusturuyor. `AuthProvider` session degisimini yakaliyor, sonra current profile bilgisini `profiles` tablosundan cekiyor. Bu profile icindeki role bilgisi frontend'de route ve menu davranislarini belirliyor.

## 10. Frontend kullanicinin rolunu nasil biliyor?

### Cevap:

Frontend direkt Supabase Auth metadata'sina guvenmiyor. Login olan kullanicinin `auth.users.id` degeri ile `profiles` tablosundan profil kaydi cekiliyor.

Bu profile kaydinda `role` alanı var:

```
admin_hr
manager
employee
```

Frontend bu role bilgisini ekran erisimi ve menu davranislari icin kullaniyor.

## 11. `ProtectedRoute` frontend tarafinda nasil calisiyor?

### Cevap:

`ProtectedRoute`, route seviyesinde kullanicinin login olup olmadigini ve rolunun o sayfaya uygun olup olmadigini kontrol ediyor.

Kullanici login degilse login sayfasina yonlendiriliyor. Login ise ama rolu izin verilen roller arasinda degilse erisim yok mesajı gösteriliyor.

Orneğin admin sayfasına sadece `admin_hr` girebilir. Employee rolundeki kullanıcı bu route'a girerse frontend bunu engeller.

## 12. ProtectedRoute güvenlik için yeterli mi?

### Cevap:

Hayır. ProtectedRoute frontend tarafında kullanıcı deneyimini düzenler ama tek başına güvenlik sağlamaz.

Gerçek güvenlik Supabase RLS tarafında sağlanır. Frontend route guard bypass edilse bile Supabase policy izin vermiyorsa kullanıcı veriye erişemez.

Bu ayrım önemli:

```
ProtectedRoute = ekran koruması  
Supabase RLS = veri koruması
```

## 13. Login sayfasında hata ve loading durumlarını nasıl yönettin?

### Cevap:

Login işlemi async olduğu için kullanıcı formu gönderdiğinde loading state kullanılıyor. Supabase hata dönerse hata mesajı kullanıcıya gösteriliyor.

Boylece kullanıcı yanlış şifre, eksik Supabase config veya bağlantı problemi gibi durumlarda sessiz bir hata yerine anlaşılar geri bildirim alıyor.

## 14. Frontend'de TypeScript kullanmanın faydası ne oldu?

### Cevap:

TypeScript sayesinde Supabase'ten gelen verilerin uygulama içinde hangi alanlara sahip olduğunu daha net tanımladım.

Orneğin `EmployeeListItem`, `Document`, `LeaveRequest`, `SalaryRecord` gibi type'lar oluşturuldu. Bu sayede componentlerde yanlış alan kullanma, status değerlerini karıştırma veya eksik property hataları daha erken yakalanıyor.

## 15. Supabase tablolarını frontend'de nasıl temsil ettin?

### Cevap:

Her service dosyasında ilgili tabloya karşılık gelen TypeScript type'ları tanımladım.

Örnek:

```
employees tablosu -> EmployeeListItem / EmployeeDetail  
documents tablosu -> Document  
leave_requests tablosu -> LeaveRequest  
devices tablosu -> Device  
salary_records tablosu -> SalaryRecord
```

Bu type'lar frontend ile backend verisi arasında sözleşme gibi çalışıyor.

## 16. EmployeesPage veriyi nasıl alıyor?

**Cevap:**

EmployeesPage direkt Supabase sorgusu yazmıyor. Sayfa, `employeeService.getEmployees()` fonksiyonunu çağırıyor.

Bu fonksiyon Supabase `employees` tablosuna sorgu atıyor, gerekli alanları seçiyor ve isim sırasına göre listeyi donduruyor.

Yani:

```
EmployeesPage
-> getEmployees()
-> supabase.from('employees').select(...)
-> UI'da tablo olarak göster
```

## 17. DocumentsPage Supabase Storage ile nasıl çalışıyor?

**Cevap:**

DocumentsPage, dosya işlemleri için `documentService` fonksiyonlarını kullanıyor.

Upload sırasında:

```
file seçilir
-> uploadEmployeeDocument() çağrılır
-> dosya Storage'a yüklenir
-> documents tablosuna metadata insert edilir
-> liste yenilenir
```

Download sırasında:

```
documents kaydından bucket/path alınır
-> getSignedUrl() çağrılır
-> Supabase geçici signed URL üretir
-> kullanıcı dosyayı indirir
```

## 18. Dosya yükleme sırasında frontend hangi kontrolleri yapıyor?

**Cevap:**

Frontend dosya seçimi, doküman tipi ve ilgili çalışan seçimi gibi kullanıcı girdilerini kontrol eder. Dosya adı service tarafında güvenli hale getirilir. Sonra Supabase Storage'a upload edilir.

Asıl yetki kontrolü ise yine Supabase RLS ve Storage policy tarafındadır.

## 19. Download için neden direkt storage path kullanılmıyor?

### Cevap:

Çünkü storage path tek başına public erişim linki değildir. Dosyalar hassas olduğu için private tutulur.

Download için `getSignedUrl()` kullanılıyor. Bu fonksiyon kısa süreli, imzalı bir URL üretir. Böylece dosya herkese açık kalmadan kullanıcıya geçici indirme erişimi sağlanır.

## 20. Frontend'de silme işlemleri nasıl yönetiliyor?

### Cevap:

Silme işlemi ilgili service fonksiyonu üzerinden yapılıyor. Örneğin doküman silmede `deleteDocument()` çağrılıyor.

Bu fonksiyon önce `documents` tablosundan metadata kaydını silmeye çalışıyor. Gerçekten satır silindiye sonra Storage dosyasını siliyor.

Frontend ise bu işlemin sonucuna göre listeyi yeniliyor veya hata mesajı gösteriyor.

## 21. Hata yönetimini frontend'de nasıl ele alıyorsunuz?

### Cevap:

Service fonksiyonları Supabase'ten error dönerse `throw new Error(error.message)` ile hatayı yukarıya taşıyor.

Sayfa componentleri bu hataları yakalayıp kullanıcıya hata state'i olarak gösteriyor. Böylece verinin yüklenemediği, kaydedilemediği veya silinemediği durumlar kullanıcı tarafında anlaşılır hale geliyor.

## 22. Loading state neden önemli?

### Cevap:

Supabase istekleri async çalıştığı için veri gelene kadar kullanıcıya boş veya hatalı ekran göstermek yerine loading state gösteriliyor.

Projede tablolar yüklenirken skeleton row veya state box kullanıldı. Bu kullanıcı deneyimini daha profesyonel hale getiriyor.

## 23. Dashboard verileri nasıl geliyor?

### Cevap:

Dashboard farklı servislerden özet sayılar alıyor. Örneğin aktif çalışan sayısı, bekleyen izin sayısı, cihaz sayısı ve doküman sayısı gibi metrikler ilgili service fonksiyonlarıyla Supabase'ten çekiliyor.

Bu sayede Dashboard tek bir tabloya bağlı değil, farklı modüllerden özet veri alan bir giriş ekranı gibi çalışıyor.

## 24. React Router yapısını nasıl kurdunuz?

### Cevap:

Route yapisi uygulamadaki sayfaları ayırmak için kullanıldı. Login, dashboard, employees, leave requests, salary, devices, documents ve profile gibi sayfalar route olarak tanımlandı.

Korunması gereken route'lar `ProtectedRoute` altına alındı. Böylece login olmayan kullanıcı uygulama paneline giremedi.

## 25. Layout yapisi nasıl çalışıyor?

**Cevap:**

Panel ekranları ortak bir layout içinde çalışıyor. Sidebar, Header ve ana içerik alanı `AppLayout` tarafından sağlanıyor.

Bu sayede her sayfada aynı navigasyon ve panel iskeleti tekrar yazılmıyor. Sayfalar sadece kendi içeriklerine odaklanıyor.

## 26. Sidebar ve Header rol bilgisine göre değişiyor mu?

**Cevap:**

Evet, kullanıcının rolüne göre gösterilecek menu ve aksiyonlar değişebilir. Auth context icinden gelen profile/role bilgisi kullanılarak kullanıcıya uygun navigasyon deneyimi sunulur.

Ancak burada tekrar vurgulamak gerekir: menu gizlemek gerçek güvenlik değildir. Gerçek veri erişimi Supabase RLS ile korunur.

## 27. Component yapısında tekrar kullanılabilirlik nasıl sağlandı?

**Cevap:**

Ortak UI parçaları `components/ui` altında toplandı. Button, Modal, Panel, Badge, StatCard, Field, Tabs gibi componentler farklı sayfalarda tekrar kullanıldı.

Bu sayede tasarım tutarlılığı sağlandı ve aynı UI kodu tekrar tekrar yazılmadı.

## 28. Form işlemlerinde nelere dikkat ettin?

**Cevap:**

Formlarda kullanıcı girdileri state ile takip edildi. Kaydetme işlemleri async service fonksiyonlarıyla yapıldı. Başarılı olursa liste veya detay verisi yenilendi. Hata olursa kullanıcıya mesaj gösterildi.

Özellikle çalışan oluşturma, cihaz atama, izin talebi oluşturma ve doküman yükleme gibi işlemlerde form state'i ile backend işlemi ayrıldı.

## 29. Frontend'de optimistic update kullandın mı?

**Cevap:**

Genel olarak kritik işlemlerde Supabase sonucunu bekleyen daha güvenli bir yaklaşım kullandım. Özellikle silme, upload ve update gibi işlemlerde backend başarılı olmadan UI'nin kesin olarak değişmesi riskli olabilir.

Bu MVP için veri tutarlılığını önceleyen bir yaklaşım tercih ettim.

### 30. UI ile RLS hatalari arasindaki iliskiye nasil yonettin?

#### Cevap:

RLS nedeniyle bir sorgu hata donebilir veya hic satir donmeyebilir. Service fonksiyonlari bu durumlari hata olarak yukariya tasiyor. Frontend de kullaniciya yetki veya islem hatasi mesaji gosteriyor.

Ornegin dokuman silmede satir silinmediyse kullaniciya "Bu kayit icin silme yetkiniz olmayabilir" anlaminda mesaj veriliyor.

### 31. Supabase baglantisi yoksa frontend ne yapiyor?

#### Cevap:

`supabaseClient.ts` icinde env degerleri yoksa `supabase` null oluyor. Service fonksiyonlari bu durumda "Supabase not configured" hatasi firlatiyor.

Bu sayede eksik config durumunda uygulama sessizce bozulmak yerine gelistiriciye anlasilir hata veriyor.

### 32. `.env.local` neden Git'e eklenmemeli?

#### Cevap:

`.env.local` icinde proje URL'si, anon key ve test kullanici bilgileri gibi ortama ozel degerler bulunabilir. Bu dosya Git'e eklenmemelidir.

Ozellikle service role key, database sifresi veya test kullanici sifreleri kesinlikle repository'ye commit edilmemelidir.

### 33. Frontend tarafinda CSV export nasil dusunuldu?

#### Cevap:

Maaş gibi kayitlarda raporlama ihtiyaci oldugu icin CSV export yardimci fonksiyonu kullanildi. Veri Supabase'ten service araciligiyla cekilir, sonra frontend tarafinda CSV formatina donusturulup kullaniciya indirilir.

Bu MVP icin yeterli bir cozumdur. Daha buyuk veri setlerinde server-side export daha dogru olabilir.

### 34. React tarafinda global state manager neden kullanmadin?

#### Cevap:

Bu projede Redux veya Zustand gibi global state manager'a ihtiyac duymadim. Cunku global olmasi gereken ana state auth bilgisiydi ve bunu React Context ile yonettim.

Modul verileri sayfa seviyesinde yuklenip kullanildi. Bu MVP icin daha sade ve anlasilir bir yapi sagladi.

### 35. Frontend performansi icin ne yaptin?

#### Cevap:

Veri sorgularinda ihtiyac olan alanlari sectim, gereksiz tum kolonlari her yerde cekmemeye dikkat ettim. Loading/skeleton yapilariyla kullanici deneyimini iyilestirdim.



Dashboard gibi alanlarda da sayı hesapları için `count` sorguları kullanıldı. Bu, tüm veriyi çekip frontend'de saymaktan daha verimli bir yaklaşımdır.

### 36. Componentlerde veri çekme akisini nasıl kurdun?

#### Cevap:

Sayfa componentleri acıldığında ilgili service fonksiyonu çağrılıyor. Veri yüklenirken loading state aktif oluyor. Başarılı olursa state'e data yazılıyor, hata olursa error state'e mesaj yazılıyor.

Genel akis:

```
mount
-> loading true
-> service fonksiyonu çağrılır
-> data veya error gelir
-> loading false
-> UI güncellenir
```

### 37. Frontend'de veri filtreleme nasıl yapılıyor?

#### Cevap:

Bazı modüllerde filtreler service fonksiyonuna parametre olarak gönderiliyor. Örneğin dokümanlarda `employee_id` veya `document_type` filtresi, izin taleplerinde `status` veya `leave_type` filtresi kullanılıyor.

Service fonksiyonu bu filtrelere göre Supabase sorgusuna `.eq()` koşulları ekliyor.

### 38. Frontend ile backend arasındaki sözleşmeyi nasıl korudun?

#### Cevap:

TypeScript type'larıyla korudum. Her tablo için frontend'de type tanımladım. Bu type'lar hangi alanların geldiğini, hangi status değerlerinin kullanıldığını ve hangi fonksiyonun ne dondurdugunu netleştirdi.

Bu sayede frontend ve Supabase tablo yapısı arasında daha kontrollü bir bağ kuruldu.

### 39. Bu frontend yapısında en güçlü taraf ne?

#### Cevap:

En güçlü taraf, UI ile veri erişiminin ayrılmış olması. Sayfalar sadece görünüm ve etkileşime odaklanıyor. Supabase ile konuşan kodlar service dosyalarında toplanıyor.

Bu yapı projeyi daha okunabilir, test edilebilir ve geliştirilebilir hale getiriyor.

### 40. Bu frontend yapısında geliştirilebilecek taraf ne?

#### Cevap:

Production seviyesinde React Query veya benzeri bir data fetching kütüphanesi eklenebilir. Böylece cache, refetch, mutation state ve background update gibi konular daha profesyonel yönetilebilir.

Ayrica form validation icin Zod veya React Hook Form gibi araclar eklenebilir. Buyuk veri listelerinde pagination ve server-side filtering daha da guclendirilebilir.

#### 41. React Query kullansaydin ne kazandirirdi?

**Cevap:**

React Query kullansaydim server state yonetimi daha guclu olurdu. Loading, error, cache, refetch ve mutation durumlari daha standart hale gelirdi.

Bu MVP'de manuel state yonetimi yeterliydi ama production'da React Query iyi bir iyilestirme olurdu.

#### 42. Form validation'i production'da nasil iyilestirirdin?

**Cevap:**

Production'da React Hook Form ve Zod kullanirdim. Zod ile form schema'larini tanimlayip hem frontend validation hem de tip guvenligini daha guclu hale getirirdim.

Boylece email formatlari, zorunlu alanlar, maas degerleri, tarih araliklari gibi kontroller daha merkezi ve bakimi kolay olurdu.

#### 43. Frontend tarafinda en kritik guvenlik yanilgisi ne olurdu?

**Cevap:**

En kritik yanilgi, frontend'de buton veya menu gizlemenin gercek guvenlik oldugunu dusunmek olurdu.

Frontend sadece kullanıcı deneyimini yonetir. Gercek guvenlik backend tarafinda, bu projede de Supabase RLS policy'lerinde saglanir.

#### 44. Bu projede frontend-backend baglantisini tek cumleyle nasil aciklarsin?

**Cevap:**

React sayfalari kullanıcı arayuzunu yonetiyor, feature service dosyalari Supabase JS Client uzerinden Auth, PostgreSQL ve Storage servisleriyle konusuyor; rol ve veri guvenligi ise Supabase RLS tarafinda uygulanıyor.

#### 45. Isveren sana "Bu frontend production'a hazır mı?" derse ne cevap verirsin?

**Cevap:**

Bu haliyle MVP seviyesinde calisan, moduler ve Supabase ile entegre bir frontend. Production icin temel mimari dogru; auth, route guard, service katmani ve typed data modelleri var.

Ama production'a cikmadan once React Query, daha guclu form validation, pagination, detayli testler, hata takip sistemi ve daha kapsamli accessibility kontrolleri eklerdim.

#### 46. Frontend tarafinda test stratejin ne olurdu?

**Cevap:**

Component testleri için React Testing Library kullandım. Kritik akislar için login, role-based route access, dokuman upload/download ve çalışan CRUD senaryolarını test edtim.

End-to-end test için Playwright kullanarak kullanıcı akislerini gerçek tarayıcı ortamında doğruladım.

## 47. Supabase hatalarını kullanıcıya direkt göstermek doğru mu?

### Cevap:

Geliştirme ortamında detaylı hata mesajı faydalı olabilir. Ancak production'da Supabase'in ham hata mesajlarını direkt kullanıcıya göstermek yerine daha kontrollü ve kullanıcı dostu mesajlar kullanmak daha doğru olur.

Log tarafında detay tutulabilir, UI tarafında ise "Bu işlem gerçekleştirilemedi" gibi daha sade mesajlar gösterilebilir.

## 48. Frontend'de yetkiye göre butonları gizledin mi?

### Cevap:

Evet, kullanıcının rolüne göre aksiyonlar ve sayfa erişimleri düzenlenebilir. Örneğin employee rolünün admin operasyonlarını görmemesi gerekir.

Ama bu sadece UX içindir. Buton gizlemek tek başına güvenlik değildir. Supabase RLS yine asıl karar mekanizmasıdır.

## 49. Bu frontend mimarisini neden modüler sayıyorsun?

### Cevap:

Çünkü her iş alanı kendi dosya grubuna ayrıldı. Auth işlemleri auth feature'ında, çalışan işlemleri employees feature'ında, dokuman işlemleri documents feature'ında tutuldu.

Bu sayede yeni bir modul eklemek veya mevcut bir modulu değiştirmek daha kolay hale geldi.

## 50. Bu frontend tarafında en çok ne öğrendin?

### Cevap:

Frontend'in sadece ekran tasarımı olmadığını, backend ile sağlam bir veri akışı kurmanın da frontend mimarisinin parçası olduğunu öğrendim.

Özellikle AuthProvider, ProtectedRoute, service katmanı, TypeScript modelleri ve Supabase RLS ayırımı birlikte düşünmek bu projede benim için en önemli kazanımdı.